

YOLOv8 测试 4：在 Linux 中使用 Docker 部署 YOLOv8 模型，并使用简单的命令行脚本测试模型

原创 蟾宫曲 于 2024-04-09 12:45:57 发布 阅读量5.1k 收藏 82 点赞数 80 版权

分类专栏: YOLOv8 Docker 文章标签: YOLO docker linux centos conda 目标检测

 云原生 文章已被社区收录 [加入社区](#)

 Docker 同时被 2 个专栏收录 [5 订阅](#) [13 篇文章](#) [订阅专栏](#)

一、前言

记录时间 [2024-4-9]

系列文章简摘：

- YOLOv8 模型的简单测试，Windows 环境下安装部署（Python+PyTorch+Conda+cpu+CLI）
- YOLOv8 模型的简单测试 2，PyCharm 集成开发环境安装使用（Windows+Python+PyTorch+Conda+cpu）
- Win11中安装虚拟化软件VMware，以及Linux虚拟机的详细安装配置
- Docker 学习笔记（二）：在 Linux 中部署 Docker（Centos7 下安装 docker、环境配置，以及镜像简单使用）
- Docker 学习笔记（三）：Centos7 中 Docker 使用，镜像、容器，以及操作等常用命令小结

本文主要讲述如何在 Linux 中使用 Docker 部署 YOLOv8 模型，并使用简单的命令行脚本测试模型。

准备工作：

- Docker 官方文档
- YOLOv8 官方文档
- Linux 云服务器/虚拟机
- 远程连接工具 FinalShell

二、实现思路

- 准备 Linux 服务器/虚拟机，在 Centos7 中部署 Docker
- 参考 Windows 下，我们安装了 Conda 环境，那么在 Docker 中也部署 Conda 环境
- 下载 YOLOv8 模型
- 进行简单的命令行脚本测试

整理完思路是不是发现变简单啦？动手操作一下吧。

三、部署 Docker

- 在 Centos7 中部署 Docker
- 准备 Linux 服务器/虚拟机，虚拟机安装详细步骤，参考[这篇文章](#)
 - 在 Centos7 中部署 Docker，参考[这篇文章](#)
 - 了解 Docker 的常用命令和简单使用，参考[这篇文章](#)

查看下 Docker 版本，检查是否安装成功：

```
1 | docker version
```

四、部署 Conda

在 Docker 中部署 Conda 环境

1. 下载 Conda 镜像

使用 Docker 搜索 `miniconda3`：

```
1 | docker search miniconda3
```

查看结果：

```
1 | [root@localhost ~]# docker search miniconda3
2 | NAME                                DESCRIPTION                                STARS
3 | datajoint/miniconda3                A minimal base docker image with conda.    2
4 | continuumio/miniconda3              Powerful and flexible package manager       475
5 | conda/miniconda3                    Ready to use, debian-based, miniconda3 docke... 53
```

我们选择 `conda/miniconda3` 镜像下载：

```
1 | docker pull conda/miniconda3
```

2. 创建容器并运行

`-it` 是交互式方式，运行命令后，我们能直接进入 `yolopy02` 容器

```
1 | docker run -it --name yolopy02 conda/miniconda3 /bin/bash
```

3. 配置 Conda 环境

进入 `yolopy02` 容器后，创建虚拟环境：

```
1 | conda create -n yolov8 python=3.9
2 | # Proceed ([y]/n)? y
```

创建完成检查一下：

```
1 | conda env list
```

检查结果：

```
1 | root@5267608bda3f:/# conda env list
2 | # conda environments:
3 | #
4 |
```

```
5 | base * /usr/local
   | yolov8 /usr/local/envs/yolov8
```

然后进入 **base** 环境

```
1 | source activate
```

4. 激活 Conda 环境

创建 **yolov8** 运行:

```
1 | conda activate yolov8
```

此时进入 yolov8 成功:

```
1 | (base) root@5267608bda3f: /# conda activate yolov8
2 | (yolov8) root@5267608bda3f: /#
```

5. 配置清华源镜像

升级 **pip** 到最新的版本 (>=10.0.0) 后进行配置:

想了解更多, 点击[这里](#)

```
1 | pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
```

6. 安装 PyTorch

进入[官网](#), 在里面找合适的版本, 文中使用的是 **Windows** 下的 **CPU only** 版本, 没有特殊要求的话, 直接复制安装即可。

注意: 命令中的斜杠 \ 是在一条命令没写完但是要换行情况下使用, 如果命令写在同一行就不需要斜杠。

```
1 | pip install torch==1.13.1+cpu torchvision==0.14.1+cpu torchaudio==0.13.1 \
2 | --extra-index-url https://download.pytorch.org/whl/cpu
```

五、部署 YOLOv8 模型

此处使用 pip 方式, 更多方式请参考[YOLOv8 官方文档](#)

1. 下载 YOLOv8 模型

检查下 Python 环境是否正确, 我们下载的是 3.9 版本

```
1 | python --version
```

下载 YOLOv8 模型

```
1 | pip install ultralytics
```

查看下载信息可知，`ultralytics` 下载在 `/usr/local/envs/yolov8/lib/python3.9/site-packages` 目录中，进入该目录。

```
1 | cd /usr/local/envs/yolov8/lib/python3.9/site-packages/ultralytics
```

2. 重置 YOLO

如果之前有其他项目使用过 YOLO，可能会导致 YOLOv8 训练时生成的 `runs` 文件不在项目根目录中而在其他项目路径中的问题。

解决办法是重置一下：

```
1 | yolo settings reset
```

呜呜呜，遇到报错了

```
1 | ImportError: libGL.so.1: cannot open shared object file: No such file or directory
```

原因分析：所用的 Docker 镜像中，Opencv 所需依赖 `libGL.so.1` 缺失

解决一下吧，更新下 `opencv-python` 依赖

```
1 | pip uninstall opencv-python -y
2 | pip install opencv-python-headless -i https://pypi.tuna.tsinghua.edu.cn/simple
```

再试试重置，成功啦！

六、命令行脚本测试

好啦，进入测试环节。

参考参考YOLOv8 官方文档，用命令行脚本（CLI）测试一下：

```
1 | yolo predict model = yolov8n.pt source=assets/bus.jpg
```

- `bus.jpg` 是模型自带的测试图片，存放在 `ultralytics/assets/` 目录下
- 用 `yolov8n.pt` 模型对图片进行目标识别

识别结果如下：

```
1 | # 第一次使用 yolov8n.pt 需要下载，它会从官网下载
2 | Downloading https://github.com/ultralytics/assets/releases/download/v8.1.0/yolov8n.pt to 'yolov8n.pt'...
3 |
4 | # 下载完成后，对图片进行目标识别
5 | # 识别结果：图片中有 4 persons, 1 bus, 1 stop sign
6 | image 1/1 /usr/local/envs/yolov8/lib/python3.9/site-packages/ultralytics/assets/bus.jpg: 640x480 4 perso
7 | Speed: 12.0ms preprocess, 302.0ms inference, 231.8ms postprocess per image at shape (1, 3, 640, 480)
8 |
9 | # 识别结果保存，保存形式为图片
10 |
11 |
```

七、下载测试结果

因为我们使用了 FinalShell，它有一个非常好用的功能，那就是：**下载图片到本地**。

我们尝试把识别结果下载下来。

1. 退出容器

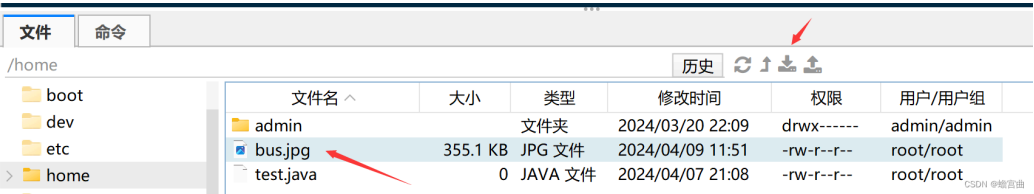
```
1 | exit
```

2. 把容器内文件复制到宿主机

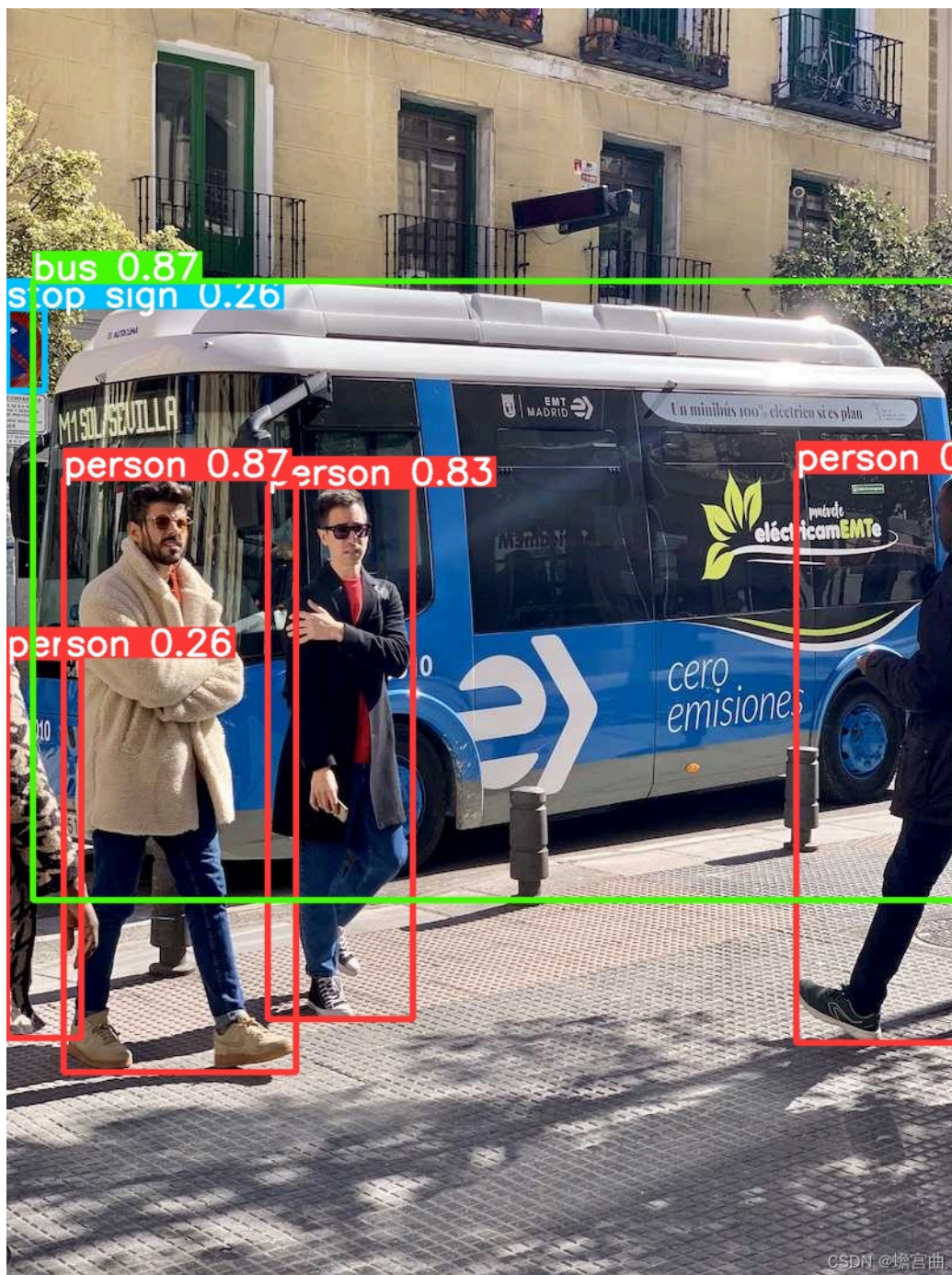
```
1 | docker cp 5267608bda3f:/usr/local/envs/yolov8/lib/python3.9/site-packages/ultralytics/runs/detect/predic
```

3. 下载图片到本地

如图，在宿主机 home 目录下找到 bus.jpg，选择下载即可**在本机桌面查看图片**。



识别结果长这样子：



八、总结

本文主要讲述如何在 Linux 中使用 Docker 部署 YOLOv8 模型，并使用简单的命令行脚本测试模型。

一些参考资料

Docker 官方文档: <https://docs.docker.com/engine/install/centos/>

Docker 远程仓库: <https://hub.docker.com/>

FinalShell 下载: <http://www.hostbuf.com/t/988.html>

YOLOv8 官方文档: <https://docs.ultralytics.com/zh/>

YOLOv8 模型仓库地址: <https://github.com/ultralytics/ultralytic>